

# Secure Art Gallery Web Application

## Phase I Requirements Report

BIBIFI Group 11

Huseyin Simsek and Gustavo Pepe

Kent State University

CS 4/53401 Secure Programming

Fall 2026 | Prepared October 3, 2026

### Purpose and scope

We are building a small web application that records guests and employees entering and leaving an art gallery and its rooms. This report defines the functional, business-logic, and security requirements for our two-person project. Our current milestone is the Week 6 backend and database, with basic demonstration pages and additional gallery controls included early.

### Submission information

The main project specification is F26\_BIBIFI\_Project.pdf. The Phase I handout specifies a PDF requirements report. Our GitHub repository is <https://github.com/hsimsek1/secure-art-gallery-group11>; the local project is named secure-art-gallery.

The Phase I handout carries a Fall 2024 label and a September 25 due date. We use its content requirements and the main specification's Fall 2026 project schedule. The current Canvas submission date requires confirmation.

# Project overview and users

The application runs locally in a browser, with Flask handling requests, SQLAlchemy managing parameterized database operations, and SQLite storing persistent records. Login accounts and tracked persons are separate: a staff member can record movements for a visitor or employee who has no login account. Seed records are fictional.

## Technology stack

Python 3.12; Flask; Flask-SQLAlchemy and SQLAlchemy; SQLite; Flask-Login; Flask-WTF for CSRF protection; Jinja with HTML/CSS; Werkzeug script password hashing; python-dotenv; Pytest. Waitress provides an optional local WSGI server. No separate frontend framework or REST service is needed.

## Users and roles

| Role          | Authorized actions                                                                                          | Restricted information                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| User or Guest | Log in, log out, view gallery and room occupancy totals                                                     | No person names, movement history, event writes, user management or audit logs               |
| Employee      | Guest permissions plus register persons, record movements, view named occupancy and search event history    | No user or role management; no audit log access                                              |
| Administrator | Employee permissions plus create accounts, change other users' roles and active status, and view audit logs | Cannot change own role or disable own account; cannot edit or delete event and audit history |

## Functional requirements

| ID    | Requirement and acceptance criterion                                                                                 |
|-------|----------------------------------------------------------------------------------------------------------------------|
| FR-01 | Store users, persons, rooms, gallery events and audit logs in a relational database; data survives restarting Flask. |
| FR-02 | Provide password login and POST logout; unauthenticated visitors are redirected from protected pages.                |
| FR-03 | Let authorized staff register guest and employee persons and record all four required movement types.                |
| FR-04 | Show current gallery occupancy including rooms and per-room occupancy from the committed event ledger.               |
| FR-05 | Let staff filter paginated history by person, room, event type and inclusive UTC date range.                         |
| FR-06 | Let administrators create accounts and manage other users' roles and active status; record each change.              |
| FR-07 | Provide login, dashboard, event form, persons, history, user administration and audit pages.                         |

## Business logic and data integrity

Each person is either outside, in the gallery lobby, or in one room. A gallery entry starts a visit; a gallery exit ends it. Room occupants are included in gallery occupancy. Application users have no direct database access through the interface.

| ID    | Required invariant                                                                                                                                                       |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BL-01 | ENTER_GALLERY is permitted only when the person is outside. A duplicate entry is rejected.                                                                               |
| BL-02 | LEAVE_GALLERY requires the person to be inside the gallery and in no room. Leaving while outside is rejected.                                                            |
| BL-03 | ENTER_ROOM requires a valid room, a person inside the gallery, and no current room.                                                                                      |
| BL-04 | LEAVE_ROOM requires the person to occupy the exact room named in the request.                                                                                            |
| BL-05 | A person cannot occupy multiple rooms; room occupancy cannot exceed capacity.                                                                                            |
| BL-06 | All referenced persons and rooms must exist. Gallery events have no room ID; room events require one.                                                                    |
| BL-07 | The server assigns event order, timestamps and actor identity. Clients cannot submit these protected fields.                                                             |
| BL-08 | State validation, the event insert and its success audit are one serialized transaction. A rejected event changes no movement state and creates a rejection audit entry. |
| BL-09 | A submitted ledger version must match the current version. Stale forms and replayed requests return a conflict and require refresh.                                      |
| BL-10 | Event and audit history have no application edit/delete endpoints. Staff permissions are rechecked before a protected write.                                             |

## Data model

Users store identity, a password hash, a role, active status, creation time and a session version. Persons store name, guest/employee type and creation time. Rooms store a name and positive capacity. GalleryEvents reference a person, optional room and recording user. AuditLogs reference an optional user and store action, bounded details, UTC time and outcome.

## Development and acceptance

The project provides a virtual environment setup, pinned direct dependencies, non-destructive database initialization, idempotent reference seeding, locally generated demo credentials and automated tests. A browser demonstration must show a login, a persisted movement, current occupancy, a forbidden employee admin request and an authorized administrator request. See the README and demonstration guide.

# Functional security requirements

IMPLEMENTED means a corresponding control exists in the delivered code. It does not mean a complete penetration test, production deployment or instructor oracle run has been completed.

| ID     | Requirement                                                                                                                                                          | Current status                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| SEC-01 | Store salted, one-way script password hashes; verify with Werkzeug. Never store plaintext passwords in the database.                                                 | IMPLEMENTED                                       |
| SEC-02 | Enforce authentication and roles on every protected route; reload role and active status from the database.                                                          | IMPLEMENTED                                       |
| SEC-03 | Reject disabled accounts and expired/invalid sessions. Clear session data on login; invalidate sessions on logout and administrative account changes.                | IMPLEMENTED                                       |
| SEC-04 | Validate allowed fields, enums, lengths, IDs, dates and state transitions on the server. Reject forged actor, timestamp and role fields.                             | IMPLEMENTED                                       |
| SEC-05 | Use SQLAlchemy bound operations; autoescape user text through Jinja; prohibit dynamic SQL and unsafe HTML output.                                                    | IMPLEMENTED                                       |
| SEC-06 | Require CSRF tokens for POST requests, including login and logout. GET endpoints must not change business records.                                                   | IMPLEMENTED                                       |
| SEC-07 | Audit login success/failure, logout, gallery acceptance/rejection, person creation, user changes and practical authorization denials. Never log passwords or tokens. | IMPLEMENTED                                       |
| SEC-08 | Generate SECRET_KEY from at least 32 random bytes, encoded as hex in an ignored local .env. Exclude credentials, database and .env from Git and image builds.        | IMPLEMENTED                                       |
| SEC-09 | Use HttpOnly, SameSite=Strict and a 30-minute session lifetime; enforce Secure cookies and HSTS when APP_ENV=production.                                             | IMPLEMENTED configuration; deployed HTTPS PLANNED |
| SEC-10 | Return generic errors, bound request size and history pages, and throttle failed logins by source address.                                                           | IMPLEMENTED; comprehensive load defense PLANNED   |
| SEC-11 | Restrict guests to aggregate occupancy; restrict identifiable history to staff and audit history to administrators.                                                  | IMPLEMENTED                                       |
| SEC-12 | Encrypt transport with HTTPS for a network deployment and protect stored data with host encryption, restricted file access and protected backups.                    | PLANNED; local SQLite is not encrypted            |

The local demo uses HTTP only on loopback and therefore does not set Secure cookies. Password hashing is not reversible encryption. Current SQLite data and .env contents are not encrypted by the application. No production protection is claimed for them.

# Security goals and risk priorities

Confidentiality: prevent guests from reading identifiable movements, credentials or administrative information. Integrity: preserve valid movement states and authorized user changes. Availability: reject abusive input and support recovery. Accountability: associate protected changes with the authenticated actor and retain a reviewable history.

## Risk assessment

Likelihood and impact use Low, Medium and High before controls. P1 items are release blockers for the local milestone; P2 items require follow-up before real network use.

| Risk                                 | Likelihood / impact | Priority | Controls and residual risk                                                                |
|--------------------------------------|---------------------|----------|-------------------------------------------------------------------------------------------|
| Account impersonation                | High / High         | P1       | SEC-01, 03, 08, 10; stolen browser cookies and distributed guessing still require review. |
| Unauthorized admin or history access | High / High         | P1       | SEC-02, 04, 11; test every new endpoint as each role.                                     |
| Invalid or concurrent movements      | High / High         | P1       | BL-01 through BL-10; database/host operators remain trusted.                              |
| SQL injection or stored XSS          | Medium / High       | P1       | SEC-04, 05, 09; regressions remain possible when adding queries or templates.             |
| CSRF and replay                      | Medium / High       | P1       | SEC-03, 06 and BL-09; no claim of protection after endpoint compromise.                   |
| Service or disk exhaustion           | Medium / High       | P2       | SEC-10; unbounded historical storage and full event replay remain scale limitations.      |
| Loss or theft of the database        | Medium / High       | P2       | SEC-12 and NFR-02; backups, host encryption and restore verification remain planned.      |

## Non functional security requirements

| ID     | Requirement and verification target                                                                                                                             | Status                                                          |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| NFR-01 | Keep security checks active under load; cap forms at 16 KiB and history pages at 25 rows. Establish a measured load budget before deployment.                   | Bounds IMPLEMENTED; load test PLANNED                           |
| NFR-02 | Before network deployment, create protected daily backups and demonstrate restoration to a separate database. Target at most one day of lost data.              | PLANNED                                                         |
| NFR-03 | Use labeled forms and clear rejection messages; do not rely on hidden links or browser validation for security.                                                 | IMPLEMENTED                                                     |
| NFR-04 | Keep security-relevant operations understandable in small modules; rerun regression tests after each change and review dependency advisories before submission. | Tests IMPLEMENTED; recurring review PLANNED                     |
| NFR-05 | Use synthetic data for class demonstrations. Confirm purpose, access, retention and deletion rules before collecting real visitor information.                  | Synthetic seed IMPLEMENTED; real-data policy PLANNED            |
| NFR-06 | Protect audit retention and review failures; document that application append-only logs are not tamper-proof against a host administrator.                      | Read-only UI IMPLEMENTED; external retention/monitoring PLANNED |

# Compliance considerations and delivery

This is a local classroom prototype using fictional visitor data. It does not process payment-card or health information. We make no blanket claim of GDPR, HIPAA or PCI DSS compliance. Before real use, the team must determine which privacy laws, institutional policies and contracts apply, limit data collection, define retention, and arrange appropriate access and deletion procedures.

HIPAA rules apply to covered entities and business associates; this gallery scenario does not establish that status [3]. PCI DSS concerns entities handling payment-card data or affecting the cardholder-data environment; payment processing is outside our scope [4]. GDPR applicability depends on the actual organization and processing context, including relevant EU connections; it must be assessed before collecting real visitor data [5]. NIST SSDF is development guidance, not a certification claimed by this project [6].

## Repository and team responsibilities

Team: Huseyin Simsek and Gustavo Pepe, BIBIFI Group 11. Both members must review and explain the authentication flow, role checks, event transaction, data model and test results. Gustavo's draft is the basis for the secure design document; this report records the agreed application scope without assigning unconfirmed individual implementation contributions.

Repository: <https://github.com/hsimsek1/secure-art-gallery-group11>. Local folder: secure-art-gallery. Required layout includes src/backend, src/frontend, database/schema.sql, database/seed.sql, tests/functional, tests/security, docs/design\_document.pdf, docker-compose.yml, README.md and .gitignore. Collaborator access and Canvas upload remain submission tasks.

## Milestone boundary

Week 4 requirements and local setup are prepared; the GitHub repository exists, but collaborator access and Canvas submission remain. Week 5 design and requirements are documented. Week 6 backend/database implementation is provided with targeted tests and demonstration pages. Broader Week 8 security exercises, Week 9 instructor oracle/deployment verification, Week 10 authorized breaking and Week 11 fixes are future work.

## References

[1] F26\_BIBIFI\_Project.pdf, supplied course specification, especially sections 1, 2, 3, 5 and 6; pages 1-7.

[2] Project\_Phase1-1.pdf, supplied Phase I requirements handout, pages 1-3; content requirements applied, historical date flagged for confirmation.

[3] HHS, Covered Entities and Business Associates.  
<https://www.hhs.gov/hipaa/for-professionals/covered-entities/index.html>

[4] PCI Security Standards Council, PCI Data Security Standard.  
<https://www.pcisecuritystandards.org/standards/pci-dss/>

[5] European Commission, Application of the GDPR.  
[https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/application-gdpr\\_en](https://commission.europa.eu/law/law-topic/data-protection/information-business-and-organisations/application-gdpr_en)

[6] NIST SP 800-218, SSDF version 1.1. <https://csrc.nist.gov/pubs/sp/800/218/final>